5-2 · Docker Compose로 Spark 마스터-워커 구축 · 이론 1H

Spark 클러스터 배포 구조 이해

마스터·워커·드라이버·익스큐터 — 누가 무엇을 하나

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 마스터·워커·드라이버·익스큐터의 역할을 구별한다

② 잡을 제출하면 벌어지는 여섯 단계를 순서대로 말한다

③ Standalone·YARN·Kubernetes 는 --master 만 바뀜을 안다

④ 포트 세 개 7077·8080·4040 의 쓰임을 안다

다음 1.5H 실습에서 compose 파일 하나로 마스터 1 + 워커 2 를 실제로 띄웁니다

네 가지 역할 — 하는 일 · 몇 개 · 언제 사나

역할	하는 일	몇 개	언제 사나
마스터	자원을 관리하고 배치를 결정	1개 (이중화 가능)	클러스터가 사는 동안
워커	자기 장비의 자원을 마스터에 등록	여러 개	클러스터가 사는 동안
드라이버	내 코드가 도는 곳 · 계획을 세움	애플리케이션당 1개	그 잡이 도는 동안만
익스큐터	실제 계산 · 워커 위에 뜸	여러 개	그 잡이 도는 동안만

5-1 주방 비유의 확장 — 홀 매니저(마스터)가 빈 조리대(워커)에 요리사(익스큐터)를 배치한다

워커와 익스큐터는 다르다 — 장비와 프로세스

워커 — 장비

클러스터가 사는 동안 상시 떠 있다

- 자기 장비의 코어·메모리를 마스터에 등록
- compose 의 spark-worker 서비스
- 잡이 끝나도 남는다

VS

익스큐터 — 프로세스

잡마다 워커 위에 새로 떠서 계산한다

- 마스터가 빈 워커에 띄우라고 지시
- 드라이버에 「준비됐습니다」 등록
- 잡이 끝나면 사라진다

2. 제출 흐름

잡을 제출하면 벌어지는 여섯 단계

① 제출	② 요청	③ 배치	④ 등록	⑤ 배분	⑥ 종료
spark-submit 드라이버가 뜬다	드라이버→마스터 코어 2 · 메모리 2G	마스터가 빈 워커에 익스큐터 지시	익스큐터→드라이버 「준비됐습니다」	드라이버가 태스크 익스큐터가 계산	결과를 모으고 익스큐터 정리

②에서 자원이 모자라면 ⑤로 못 간다 — 「Initial job has not accepted any resources」 경고가 그것

클러스터 매니저 3종 — 이번 주는 Standalone

매니저	특징	언제 쓰나
Standalone	Spark 내장 · 가장 단순	학습 · 작은 전용 클러스터(이번 주)
YARN	하둡 생태계의 자원 관리자	기존 하둡 클러스터가 있을 때
Kubernetes	컨테이너 오케스트레이터	요즘 신규 구축의 기본값
Mesos	(지원 종료)	—

어느 매니저든 드라이버·익스큐터 개념과 코드는 그대로 — --master 옵션만 바뀐다

3. 배포 모드

client 모드와 cluster 모드 — 오늘은 client

client 모드 — 오늘 실습

드라이버가 spark-submit 을 친 그 자리에 뜬다

- 로그가 내 터미널에 바로 나온다
- 터미널을 닫으면 잡이 죽는다
- 개발·디버깅에 알맞다

VS

cluster 모드 — 오늘은 못 씀

드라이버가 클러스터 안의 워커에 뜬다

- 터미널을 닫아도 계속 돈다
- 파이썬 앱은 Standalone 에서 불가(실측)
- YARN · Kubernetes 같은 매니저에서 쓴다

서비스 이름으로 찾고, 포트 세 개로 본다

1. **7077 — 마스터 접수 창구**
   워커와 드라이버가 접속하는 곳 — 호스트명 spark-master 는 compose 서비스 이름

2. **8080 — 마스터 웹 UI**
   워커 목록과 애플리케이션 이력을 본다 — 4-4 의 Trino 도 8080 을 쓴다

3. **4040 — 실행 중인 잡 UI**
   5-1 에서 본 그 화면 — 잡이 끝나면 사라진다

4주차 Trino 가 떠 있으면

8080 충돌 시

8081:8080

5. 자원 배분

자원을 어떻게 나눌 것인가 — 옵션 네 개

옵션	뜻	주의
--executor-memory	익스큐터 하나가 쓸 메모리	워커 메모리보다 크면 배치 안 됨
--executor-cores	익스큐터 하나가 쓸 코어 수	—
--total-executor-cores	이 잡이 쓸 전체 코어	Standalone 에서 쓰는 옵션
--driver-memory	드라이버 메모리	collect() 를 쓸 때 중요

기본값은 「워커의 자원을 최대한 쓴다」 — 여러 잡을 동시에 돌릴 때는 --total-executor-cores 로 제한

정리

이 1시간에서 꼭 가져갈 것

1. 워커는 상시 떠 있는 장비, 익스큐터는 잡마다 뜨는 프로세스다
2. 제출 → 마스터 자원 요청 → 워커에 익스큐터 배치 → 태스크 배분
3. 자원이 모자라면 잡이 대기 상태로 멈춘다
4. Standalone·YARN·Kubernetes 는 --master 만 바뀌고 코드는 그대로다
5. 포트 세 개: 7077(접수) · 8080(마스터 UI) · 4040(잡 UI)

5-2 · 이론 1H 마무리

다음은 클러스터 구축 실습 (1.5H)

compose 파일 하나로 마스터 1 + 워커 2 를 띄우고, 워커가 마스터에 등록되는 것을 로그와 UI 로 확인합니다

- ✓ **compose 파일 작성 · up -d · ps**
- ✓ **로그에서 Registering worker 찾기**
- ✓ **--scale 로 워커 늘렸다 줄이기**
- ✓ **마스터를 껐다 켜기 (일부러 실패)**

준비할 것

~/lakehouse stop · 8080 비우기